

Trabalho Prático de Programação no Servidor

Board



● Não iniciada 0

● Em andamento 1

● Concluído

 Sprint 1 - Especificação da API em OpenAPI

GREENHERB: Plataforma Web de Gestão Inteligente de uma Estufa

GREENHERB: Plataforma Web de Gestão Inteligente de uma Estufa

1. Enquadramento
2. Objetivo do Trabalho
-
4. Regras de Negócio
5. Requisitos Funcionais Mínimos
6. Requisitos Técnicos Obrigatórios
7. Modelo de Domínio Mínimo
8. Entregáveis
9. Critérios de Aceitação do Épico
10. Regras de Funcionamento e Avaliação

1. Enquadramento

A empresa GREENHERB pretende informatizar o sistema de gestão de uma estufa dedicada à produção de ervas aromáticas, como manjerição, hortelã, alecrim e tomilho. O objetivo é apoiar o planeamento, a monitorização e o controlo das condições de cultivo e das operações associadas.

A aplicação será utilizada por técnicos da estufa, por responsáveis técnicos e por administradores. Parte da utilização ocorre em contexto operacional, podendo haver conectividade instável; por esse motivo, a solução deverá explorar mecanismos de armazenamento no browser.

2. Objetivo do Trabalho

Pretende-se o desenvolvimento de uma aplicação web que permita gerir o ciclo de vida de lotes de cultivo, planos de cultivo, medições ambientais, alertas e intervenções operacionais, garantindo autenticação, rastreabilidade e documentação da API.

4. Regras de Negócio

1. Cada plano de cultivo está associado a uma erva aromática e pode ser de tipo **regular**, **emergência** ou **pontual**.
2. O plano **regular** corresponde ao cultivo planeado antecipadamente e deve incluir intervalos aceitáveis de temperatura, humidade e luminosidade, plano de rega, fertilização e duração prevista do ciclo.
3. O plano de **emergência** aplica-se a situações excecionais, como pragas, doenças ou condições ambientais extremas, devendo definir intervalo mínimo entre intervenções, tipo de intervenção e dosagem ou intensidade.
4. O plano **pontual** corresponde a uma intervenção única e exige autorização explícita do responsável técnico.
5. As tarefas operacionais incluem, pelo menos, **rega**, **fertilização**, **colheita** e **monitorização**, podendo ocorrer em qualquer ordem desde que respeitem as condições e horários definidos.
6. O sistema deve permitir registar e consultar o estado atual de cada **lote**, os planos associados, as tarefas pendentes e executadas e as medições ambientais.

7. O sistema deve gerar alertas quando os valores medidos estejam fora dos limites definidos no plano ou quando existam falhas de sensores, como dados em falta ou incoerentes.
 8. Os alertas devem ser classificados como **Informativo**, **Aviso** ou **Crítico**, e devem poder ser marcados como **Resolvido** ou **Ignorado**, sendo obrigatória a justificação no caso de ignorar.
 9. O sistema deve suportar automação baseada em regras, podendo **sugerir** ou **executar** ações como ativar rega, ajustar ventilação ou sugerir fertilização. Deve existir modo **Manual** e modo **Automático**.
 10. Cada lote deve possuir estado **ativo**, **concluído** ou **comprometido**, datas de início e fim reais, possibilidade de divisão parcial, registo de perdas e cálculo de produtividade.
 11. Devem existir três perfis de utilizador: **Técnico**, **Responsável** e **Administrador**.
 12. O sistema deverá manter registo de auditoria das ações realizadas, identificando quem fez o quê e quando.
-

5. Requisitos Funcionais Mínimos

1. Implementar autenticação de utilizadores e controlo de acesso por perfil.
2. Permitir gestão de utilizadores por parte do administrador.
3. Permitir registo, edição, consulta e importação de ervas aromáticas através de ficheiros **CSV** ou **Excel**.
4. Permitir criar, consultar e gerir planos de cultivo dos tipos **regular**, **emergência** e **pontual**.
5. Permitir associar planos a lotes de cultivo e consultar o estado atual de cada lote.
6. Permitir registar tarefas operacionais, consultar tarefas pendentes e marcar tarefas como executadas.
7. Permitir registar medições ambientais de temperatura, humidade e luminosidade.
8. Gerar alertas automaticamente com base nas medições e nas regras do plano de cultivo.
9. Permitir resolver ou ignorar alertas, com registo da decisão e respetiva justificação quando aplicável.

10. Suportar modo **Manual** e **Automático** para ações desencadeadas por regras.
 11. Disponibilizar histórico de medições, intervenções e alertas.
 12. Permitir comparar valores reais com valores esperados do plano.
 13. Permitir exportar relatórios em **CSV** ou **Excel**.
 14. Permitir dividir lotes, registrar perdas e calcular produtividade e tempo real versus tempo planeado.
 15. Registrar logs de auditoria sobre operações relevantes do sistema.
-

6. Requisitos Técnicos Obrigatórios

1. O frontend (extra) deve ser desenvolvido em **HTML**, **CSS** e **JavaScript**.
 2. O backend deve ser desenvolvido em **Node.js**, utilizando **Mongoose** para modelação e acesso à base de dados **MongoDB**.
 3. A autenticação deve utilizar **JWT**, com proteção de rotas e autorização por perfis.
 4. As palavras-passe não podem ser armazenadas diretamente.
 5. A API deve ser descrita em **OpenAPI 3.x**, incluindo endpoints, parâmetros, esquemas de dados, códigos de resposta e mecanismo de segurança.
 6. A interface deverá ser funcional em desktop e mobile.
 7. A equipa deverá entregar um pequeno documento de arquitetura, com diagrama ou tabela, identificando: que dados são guardados em cada mecanismo, por que motivo cada mecanismo foi escolhido, política de expiração e invalidação, estratégia de sincronização com a API, comportamento em caso de falha de rede, decisão adotada para armazenamento do token **JWT** e respetiva justificação de segurança.
 8. A utilização de **IndexedDB** deve servir para dados estruturados no cliente, como histórico recente, operações pendentes, cache local de entidades ou suporte a funcionamento degradado/offline.
 9. A **Cache API** deve ser utilizada para recursos estáticos e/ou respostas selecionadas da API, com política de atualização definida pela equipa.
-

7. Modelo de Domínio Mínimo

A solução deverá contemplar, no mínimo, as seguintes entidades: `Utilizador`, `ErvaAromática`, `PlanoCultivo`, `LoteCultivo`, `Tarefa`, `MediçãoAmbiental`, `Alerta` e `LogAuditoria`.

8. Entregáveis

1. Código-fonte completo da aplicação (Github e no Moodle após apresentação do último sprint).
 2. Documento `README` com instruções de instalação, configuração e execução.
 3. Especificação da API em ficheiro `openapi.yaml` ou `openapi.json`.
 4. Documento curto da arquitetura de armazenamento no browser.
 5. Conjunto de dados de demonstração, incluindo pelo menos um exemplo de importação por `CSV` ou `Excel`.
-

9. Critérios de Aceitação do Épico

1. Um utilizador autenticado consegue aceder apenas às operações permitidas pelo seu perfil.
 2. O sistema suporta os três tipos de plano de cultivo e a associação desses planos a lotes.
 3. O registo de medições ambientais pode originar alertas automáticos.
 4. O sistema demonstra comportamento distinto em modo `Manual` e em modo `Automático`.
 5. É possível consultar histórico, exportar relatórios e observar rastreabilidade das ações.
 6. A arquitetura de armazenamento no browser está implementada e coerente com o comportamento observado na aplicação.
 7. A implementação está alinhada com a especificação `OpenAPI`.
-

10. Regras de Funcionamento e Avaliação

1. Os grupos devem ser constituídos por `3` a `4` estudantes.
2. A avaliação é realizada semanalmente em regime de `sprints`, correspondendo cada aula prática a um sprint.

3. Apesar de existir apenas um épico, a sua implementação deve ser incremental ao longo dos sprints.
4. No início de cada sprint deve ser eleito um **Product Owner** (responsável) do grupo. Este papel é rotativo e só pode repetir depois de todos os elementos o terem desempenhado.
5. No final da aula, ou na aula seguinte à da implementação do sprint, o **Product Owner** apresenta o trabalho e responde às questões do docente.
6. Se o **Product Owner** não conseguir defender o trabalho, obtém classificação **0**. Nesse caso, outro elemento do grupo poderá defender o sprint.
7. Ao concluir cada sprint, o **Product Owner** deve garantir que os artefactos produzidos estão disponíveis no repositório git.
8. Os elementos do grupo não presentes na apresentação, obtêm a classificação **0**.
9. A nota final resulta da média das notas dos sprints:
$$\text{Nota final} = \frac{\text{(somatório das notas dos sprints)}}{\text{número de sprints}}$$
10. A nota de cada sprint é calculada por
$$\text{Nota do sprint} = V \times Q \times \text{Def}$$
, em que **V** representa o volume de trabalho produzido no sprint, **Q** a qualidade da solução e **Def** a defesa do trabalho.
11. O plágio de soluções e/ou código implica a anulação do trabalho.